AI KATA 2.0

AI Kata List

This page collects the AI kata exercises used during the training track. The first kata uses an external legacy repository. All following kata exercises use the Northstar Travel repository and dedicated branches.

Main training repository: ai-kata-internationalization

1. Legacy Modernization Kata

Modernize a legacy AngularJS form builder

Kata Description

Start from an old AngularJS-based form builder and use AI assistance to understand, repair, and then fully modernize the application. The original project is intentionally dated and may not run successfully on the first attempt.

Steps

1. Download or clone the repository: kelp404/angular-form-builder
2. Run `npm install`.
3. Run `grunt dev`.
4. Observe the failure and use AI assistance to make the existing solution work locally.
5. Explore the current application behavior and architecture.
6. Rewrite the whole solution using a modern stack.
7. Add new controls and UX improvements:
    - switch / toggle control
    - date picker control
    - dark theme
    - theme switcher

Useful Links

- Source repository: kelp404/angular-form-builder
- Grunt documentation: https://gruntjs.com

Hints

- Ask AI to explain the legacy build chain before changing it.
- First make the original solution run; then use that behavior as the acceptance baseline.
- Preserve the core form-builder workflow while modernizing the implementation.
- When rewriting, ask AI to produce a migration plan before generating code.

Desired Result

A modern form builder application that reproduces the important legacy behavior and adds the requested controls and theme switching.

2. Internationalization Kata

Add internationalization to the Northstar Travel app

Kata Description

Use AI assistance to internationalize the current React travel planning application. The application should support multiple languages and keep all user-facing text outside component implementations.

Useful Links

- Repository: ai-kata-internationalization
- Branch: `feat/i18n`

- Branch README: `README.md`
- i18next: https://www.i18next.com
- react-i18next: https://react.i18next.com

Hints

- Start by asking AI to identify all hardcoded user-facing text.
- Keep structured content, such as destinations, FAQs, guides, months, and policy text, translatable as well.
- Verify language switching, browser-language detection, persistence, and the `<html lang>` value.
- Avoid mixing translation keys directly with business logic.

Desired Result

The app supports English, Lithuanian, and Mandarin Chinese. All UI copy and structured page content is translated through the i18n layer, and the selected language persists between sessions.

3. Unit Test Coverage Kata

Raise unit test coverage to the project threshold

Kata Description

Use AI assistance to add meaningful tests for utilities, services, hooks/content helpers, pages, and components. The kata focuses on coverage, but tests should still describe real behavior rather than only exercising lines.

Useful Links

- Repository: ai-kata-internationalization
- Branch: `unit-test-coverage`
- Branch README: `README.md`
- Vitest: https://vitest.dev
- React Testing Library: https://testing-library.com/docs/react-testing-library/intro

Hints

- Ask AI to inspect uncovered files before writing tests.
- Prefer behavior-focused tests over implementation-detail assertions.
- Cover pure business logic first because it is fast and stable.
- Then cover representative component and page flows with React Testing Library.
- Run `npm run test:coverage` and use the report to close remaining gaps.

Desired Result

The project passes the configured coverage gate: 95% lines, functions, statements, and branches.

4. Skills Kata

Create reusable AI agent skills

Kata Description

Use AI assistance to extract repeatable workflows into reusable agent skills. The training repository contains examples for conventional commits and unit test generation, with separate branches for Claude Code and Cursor-style skills.

Useful Links

- Repository: ai-kata-internationalization
- Claude Code skills branch: `claude-skills`
- Cursor skills branch: `cursor-skills`
- Claude skills directory: `.claude/skills`
- Cursor skills directory: `.cursor/skills`

Hints

- Choose a task that repeats often and has clear rules.
- Keep the skill focused: one skill should teach one workflow.
- Include examples and project-specific constraints.
- Test the skill by asking the agent to perform the workflow in a fresh conversation.

Desired Result

The repository contains reusable skills that help agents consistently write conventional commit messages and generate project-appropriate unit tests.

5. OpenSpec Development Kata

Use specification-driven development with OpenSpec

Kata Description

Use AI assistance to drive development from a structured specification before implementation. The goal is to practice describing the change, reviewing the proposed behavior, and then implementing against the agreed spec.

Useful Links

- Repository: ai-kata-internationalization
- Branch: `openspec-development`
- Branch README: `README.md`
- OpenSpec: https://openspec.ai

Hints

- Ask AI to draft the specification before touching code.
- Review acceptance criteria carefully and refine unclear requirements.
- Use the spec as the source of truth for implementation and tests.
- Keep generated implementation changes traceable back to the spec.

Desired Result

A feature or change is implemented from a reviewed OpenSpec-style specification, with clear acceptance criteria and verification steps.

6. MCP Kata

Use MCP tools to implement a Jira-driven feature and publish release notes

Kata Description

Configure MCP servers and use AI assistance to work with external tools such as Jira, Confluence, and browser/devtools automation. The kata task is to read a Jira ticket, implement the requested feature, test it, and publish release notes.

Useful Links

- Repository: ai-kata-internationalization
- Branch: `ai-kata-mcp`
- Kata instructions: `KATA-INSTRUCTIONS.md`
- MCP configuration: `.mcp.json`
- Jira ticket: EPMGDLT-1565
- Confluence page: AI KATA MCP
- Model Context Protocol: https://modelcontextprotocol.io

Hints

- Inspect `.mcp.json` before starting.
- Configure the required API tokens and verify the MCP tools are available.
- Ask the agent to read the Jira ticket directly through MCP.

- Use browser/devtools MCP to verify the feature in the running app.
- After implementation, ask the agent to create release notes in Confluence.

Desired Result

The Jira feature is implemented and verified. Release notes are created as a Confluence child page under the expected training location.

7. Codemie Kata

Explore Codemie assistants and integrations

Kata Description

Use Codemie to explore assistant setup, Jira integration, Confluence integration, IDE plugin usage, and assisted implementation planning. This kata focuses on comparing how Codemie can help with a Jira-driven development workflow.

Useful Links

- Repository: ai-kata-internationalization
- Branch: `discovery-codemie`
- Kata instructions: `KATA-INSTRUCTIONS.md`
- Codemie plan: `CODEMIE-PLAN.md`
- Codemie: https://codemie.lab.epam.com
- Jira ticket: EPMGDLT-1565

Hints

- Create and configure the Jira integration first.
- Build or configure a BI assistant with access to the Jira integration.
- Use the Codemie IDE plugin from VS Code while connected to VPN.
- Ask the assistant to fetch ticket details and generate an implementation plan.
- Add the Confluence knowledge base integration and use it for release notes.

Desired Result

Codemie is connected to the relevant tools, can retrieve the Jira ticket context, can help generate an implementation plan, and can support release-note creation.